Listing 4.3 Streaming the `<Chat />` component

```
export async function streamComponent(input, history) {
  'use server';
  const result = await streamUI({.
    model: openai('gpt-3.5-turbo'),
    messages: [...history, { role: 'user', content: input }],
    text: ({ content, done }) => {.
      return <ChatBubble role="assistant" text={content}
className={`mr-auto border-none`} />;
    },
  });

  return {
    id: generateId(),
    role: 'assistant',
    display: result.value,
  };
}
```

The streamUI function is called with the OpenAI model and the conversation history, including the new user input.

The text property is a function that defines how to render the AI's response. It receives the content and a done flag once the response is finished.

This function returns a ChatBubble component for each piece of content received from the AI, wrapping the AI's response in a React component.

The result.value contains the streamable UI component, which is assigned to the display property of the returned object.

The `streamComponent` server action sends the conversation history to the supported AI model and then uses the `streamUI` function to process the response. The AI's output is wrapped in a `ChatBubble` component, which can be updated in real time as more content is generated. The function returns an object with the generated response, including a unique ID and the `streamable` UI component.

We can call this action as part of the form submission process and store the results in a variable. This variable will hold the generated React component containing the LLM response, which we can then display to the user. The following listing shows how to do it.

Listing 4.4 Rendering the streamed component into the page

```
'use client';
 // imports
 export default function Page() {
  const [component, setComponent] = useState();
   const handleInputChange = (event) => {
    setInput(event.target.value);
  };
  const [input, setInput] = useState('');
  return (
    <div>
      <form
        onSubmit={async e => {
          e.preventDefault();
          const value = input.trim();
          setInput('');
          if (!value) return;
```

Initializes a state variable component to store the streamable UI component returned from the server action